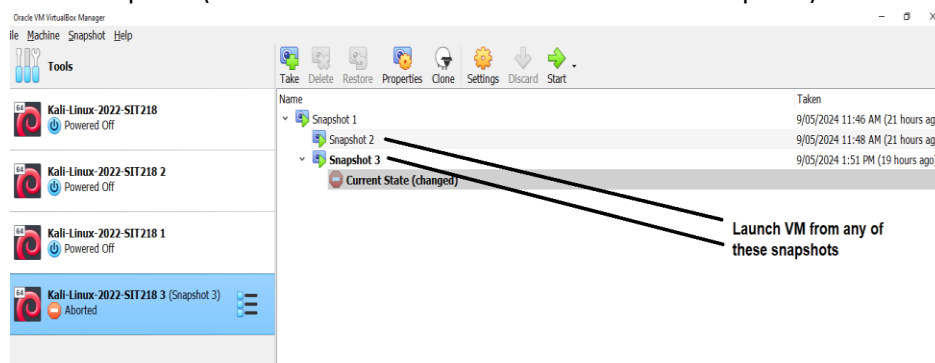


SIT 218/738: Secure coding:

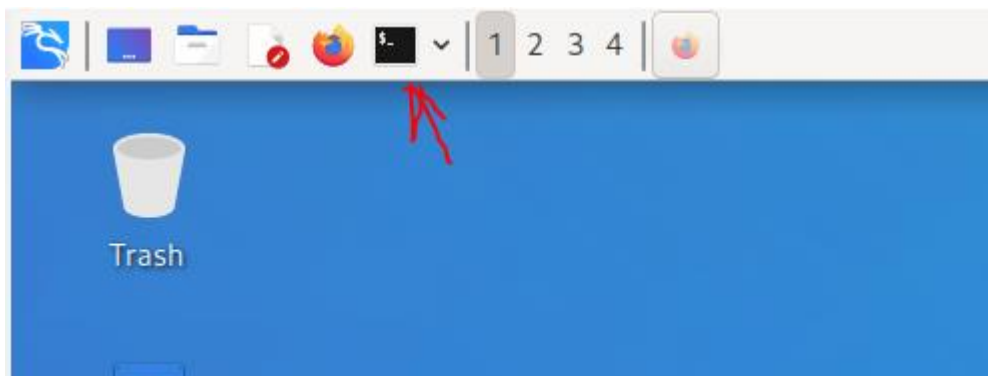
Ontrack Task2.3P Activity

Securing a web server is crucial to protect it from various threats and ensure the confidentiality, integrity, and availability of your website and data. In this workshop you will use the SimpleSocketClient java program to see the response received the Tomcat server. Furthermore, you will change the banner of the Tomcat server and understand how different configuration files are used to control the Tomcat server's settings. Follow the below steps to complete this workshop.

1. Launch the Kali-Linux-2022-SIT218 VM. Login using username and password as 'kali'
2. Take a snapshot (Virtual box main menu-> Machine ->take snapshot).



3. Then Launch the VM from the snapshot you have taken.
4. Launch a terminal from the launch bar.



5. Start Tomcat Server using the command:

```
sudo systemctl start tomcat
```

when prompted for password enter "kali"

6. Check status of the tomcat server

```
sudo systemctl status tomcat
```

```
(kali@kali)-[~]
$ sudo systemctl status tomcat
● tomcat.service - Tomcat 9.0 servlet container
   Loaded: loaded (/etc/systemd/system/tomcat.service; disabled; vendor preset: enabled)
   Active: active (running) since Tue 2023-06-13 17:29:50 EDT; 5s ago
     Process: 8152 ExecStart=/opt/tomcat/latest/bin/startup.sh (code=exited, status=0/SUCCESS)
    Main PID: 8162 (java)
   CGroup: /system.slice/tomcat.service
           └─8162 /usr/lib/jvm/default-java/bin/java -Djava.util.logging.config.file=/opt/tomcat/latest/conf/logging.properties -Djava.awt.headless=true -Dcom.sun.management.jmxremote.port=8161 -Dcom.sun.management.jmxremote.ssl=false -Dcom.sun.management.jmxremote.authenticate=false -Dcom.sun.management.jmxremote.username=admin -Dcom.sun.management.jmxremote.password.file=/opt/tomcat/latest/conf/jmxremote.password -Dcom.sun.management.jmxremote.local.only=false --add-exports java.desktop/com.sun.java.swing=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing.internal=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing.internal=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing.internal=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing.internal=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing.internal=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing.internal=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing.internal=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing.internal=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing.internal=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing=ALL-UNNAMED --add-exports java.desktop/com.sun.java.swing.internal=ALL-UNNED

Jun 13 17:29:50 kali systemd[1]: Starting Tomcat 9.0 servlet container...
Jun 13 17:29:50 kali startup.sh[8152]: Existing PID file found during start.
Jun 13 17:29:50 kali startup.sh[8152]: Removing/clearing stale PID file.
Jun 13 17:29:50 kali startup.sh[8152]: Tomcat started.
Jun 13 17:29:50 kali systemd[1]: Started Tomcat 9.0 servlet container.
```

You should see the tomcat server is active and running.

Launch eclipse

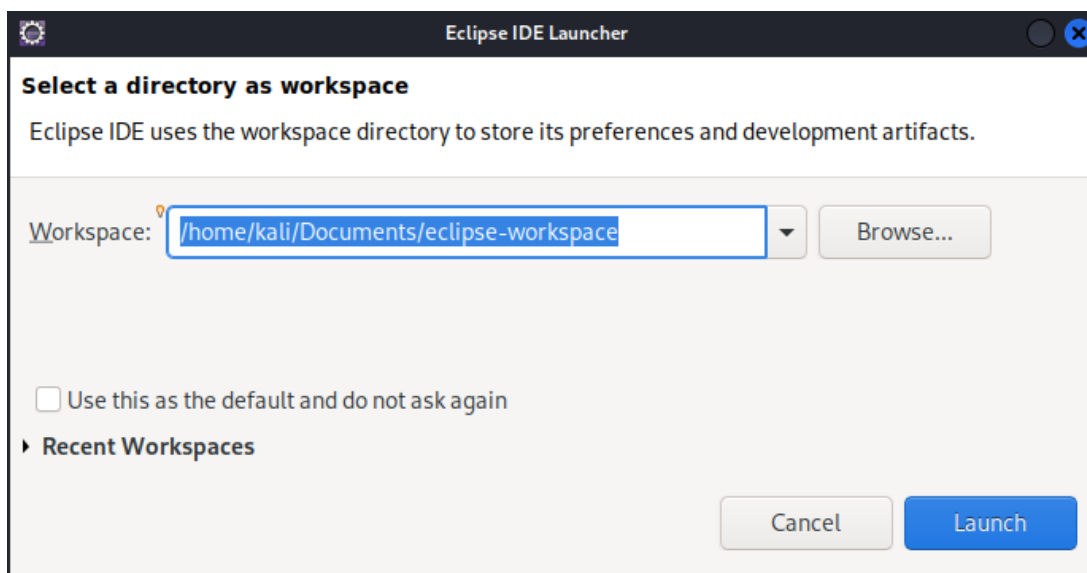
In a new terminal type:

cd /opt/eclipse/eclipse/

Then enter:

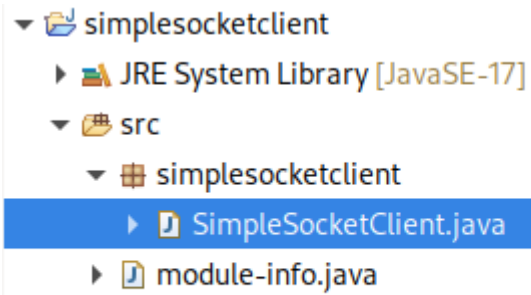
sudo ./eclipse

when prompted for password enter "kali"



Click on Launch in the Launcher window

Once the editor launches, select simplesocketclient project and access the SimpleSocketClient.java source file. Double click on it and the code will be loaded in the editor.



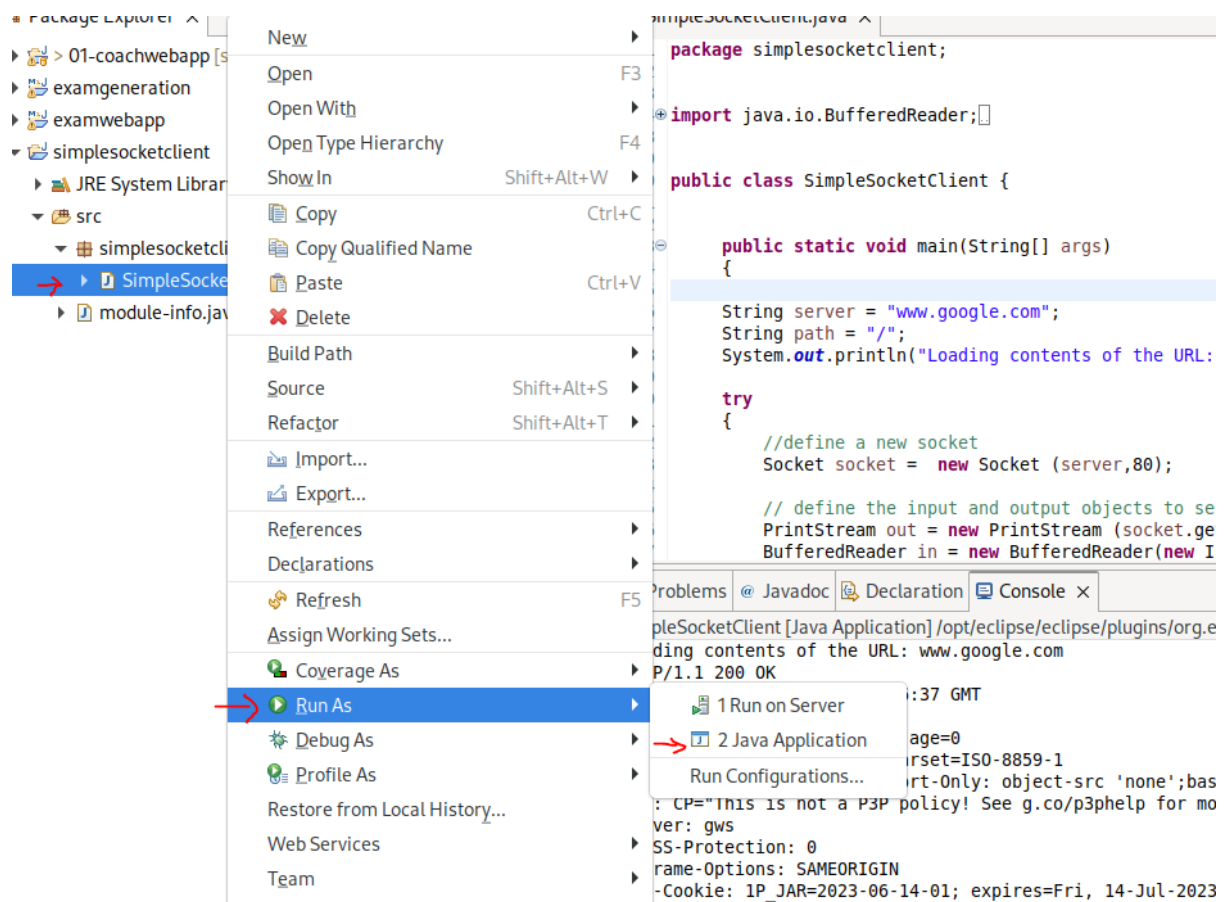
In the code make the following changes to the server (**www.google.com**) and port (**80**) as shown in the figure below:

```
1 package simplesocketclient;
2
3
4 import java.io.BufferedReader;
5
6
7
8
9
10 public class SimpleSocketClient {
11
12
13     public static void main(String[] args)
14     {
15
16         String server = "www.google.com";
17         String path = "/";
18         System.out.println("Loading contents of the URL: " + server);
19
20         try
21         {
22             //define a new socket
23             Socket socket = new Socket(server, 80);
24         }
```

Save the changes with cntrl+s.

Now let's execute the code and view the response from the google's web server.

Right click on the code in the project window and select 'Run As' -> Java Application.



Response from the google web server:

```
SimpleSocketClient [Java Application] /opt/eclipse/eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.linux.x86_64_17.0
Loading contents of the URL: www.google.com
HTTP/1.1 200 OK
Date: Wed, 14 Jun 2023 01:06:37 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=ISO-8859-1
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-LcekkfDbFD296
P3P: CP="This is not a P3P policy! See g.co/p3phelp for more info."
Server: gws
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
Set-Cookie: 1P_JAR=2023-06-14-01; expires=Fri, 14-Jul-2023 01:06:37 GMT; path=/; domain=.google.com; s
Set-Cookie: AEC=AUEFqZclRfRJZyjnrxGvt2P5SXKdlvmC0z_HKG4czo_1x5sfH_AnYm8w; expires=Mon, 11-Dec-2023 0
Set-Cookie: NID=511=I_7TwB5hRfmwV-L3cm-EZqWMrrkVt7Ys8X6Rxm0FBDrXY-msbpjx1NBspqk0xyC0afeQbs88Bbl5pxNtG1
Accept-Ranges: none
Vary: Accept-Encoding
Transfer-Encoding: chunked
```

Now lets get the response from our Apache tomcat server installed locally. For this we will change the server to localhost and port number to 8080.

```

public class SimpleSocketClient {

    public static void main(String[] args)
    {

        String server = "localhost";
        String path = "/";
        System.out.println("Loading contents of the URL: "+ server);

        try
        {
            //define a new socket
            Socket socket = new Socket(server, 8080);

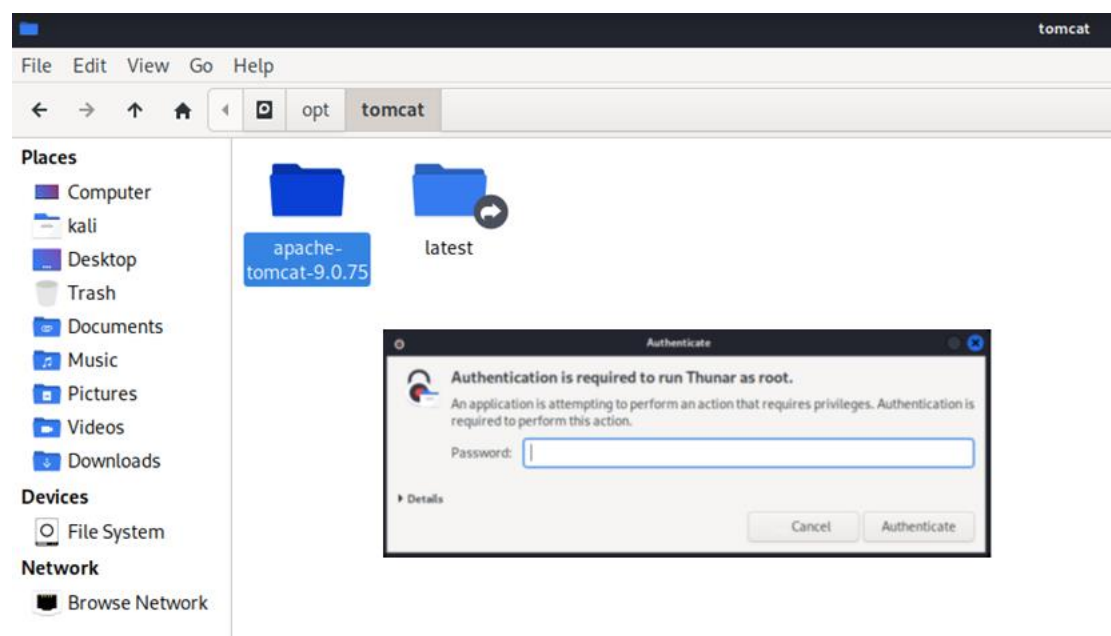
```

Now view the response of the server. You will notice that the Apache's default manage page will be sent as the response to the GET request. Now to control the response (banner) and pages reported back by the tomcat server we need to edit the server.xml file in tomcat configuration folder.

First let us stop the tomcat server using the following command:

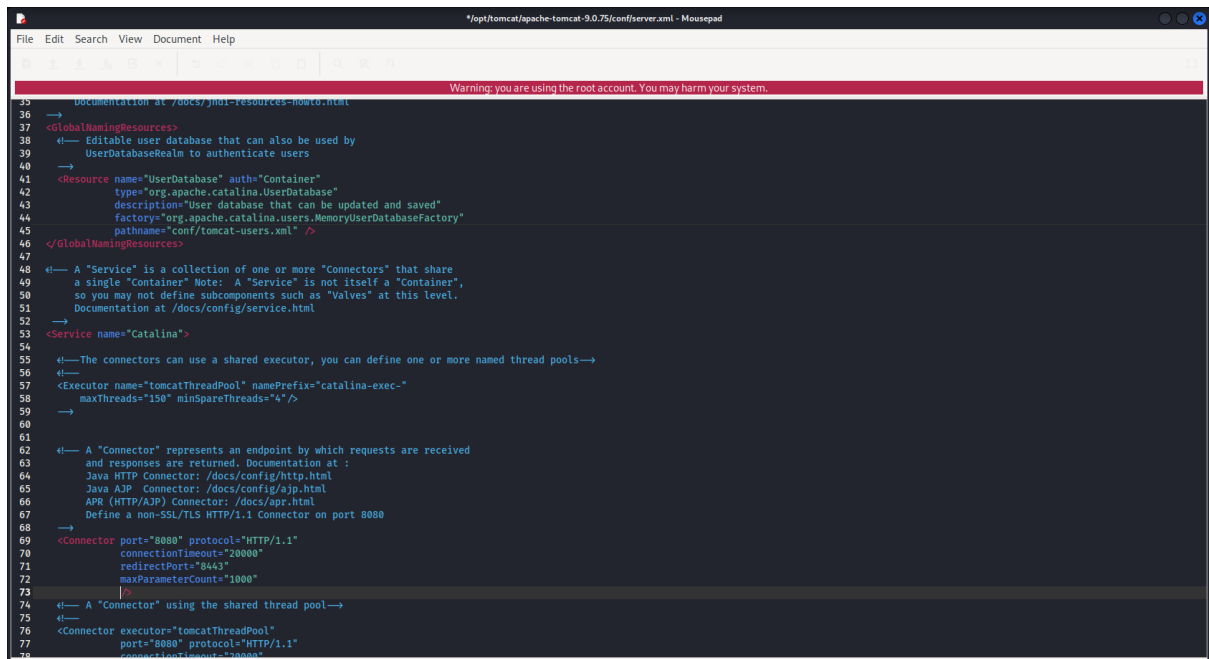
sudo systemctl stop tomcat

Open the folder **/opt/tomcat/apache-tomcat-9.0.75** as a root user from the File System.



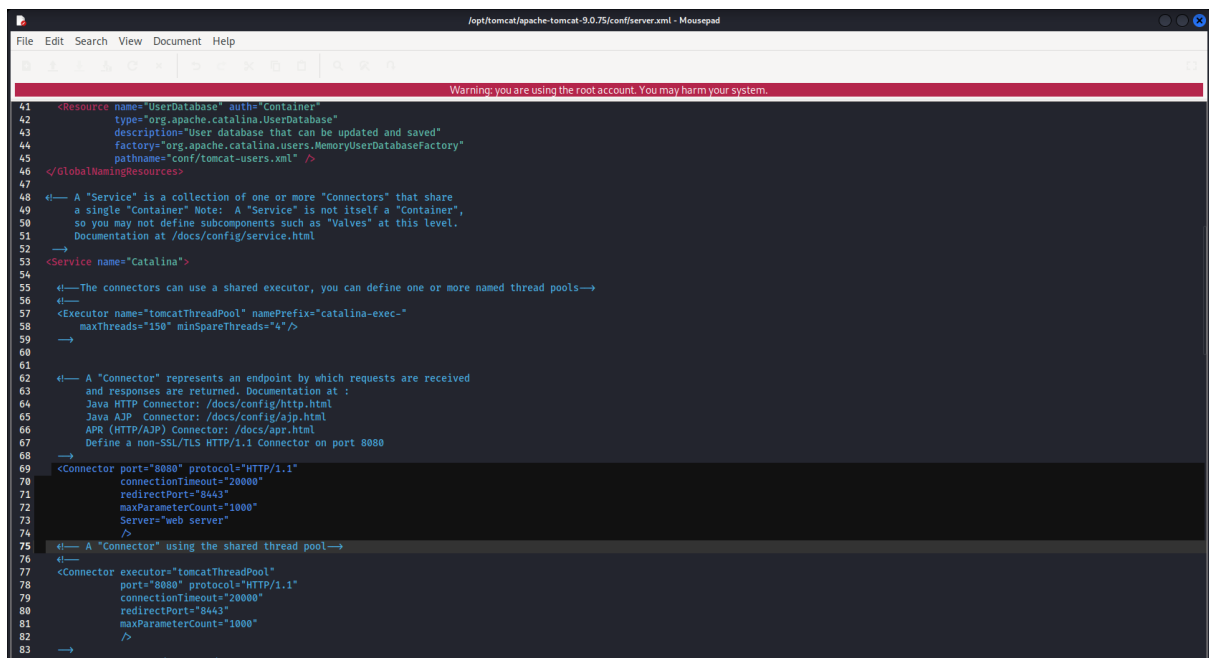
Go to the folder **/opt/tomcat/apache-tomcat-9.0.75/conf**

Now let's edit the **"server.xml"** file in the above folder (**/opt/tomcat/apache-tomcat-9.0.75/conf**) using mousepad (right click on server.xml file and open with mousepad)



```
35  Documentation at /docs/jmx-resources/mbeans.html
36  →
37  <GlobalNamingResources>
38  <!-- Editable user database that can also be used by
39  UserDatabaseRealm to authenticate users
40  →
41  <Resource name="UserDatabase" auth="Container"
42  type="org.apache.catalina.UserDatabase"
43  description="User database that can be updated and saved"
44  factory="org.apache.catalina.users.MemoryUserDatabaseFactory"
45  pathname="conf/tomcat-users.xml" />
46 </GlobalNamingResources>
47
48 <!-- A "Service" is a collection of one or more "Connectors" that share
49 a single "Container" Note: A "Service" is not itself a "Container",
50 so you may not define subcomponents such as "Valves" at this level.
51 Documentation at /docs/config/service.html
52 →
53 <Service name="Catalina">
54
55 <!-- The connectors can use a shared executor, you can define one or more named thread pools-->
56 <!--
57 <Executor name="tomcatThreadPool" namePrefix="catalina-exec-"
58 maxThreads="150" minSpareThreads="4"/>
59 →
60
61 <!-- A "Connector" represents an endpoint by which requests are received
62 and responses are returned. Documentation at :
63 Java HTTP Connector: /docs/config/http.html
64 Java AJP Connector: /docs/config/ajp.html
65 APR (HTTP/AJP) Connector: /docs/apr.html
66 Define a non-SSL/TLS HTTP/1.1 Connector on port 8880
67 →
68 <Connector port="8880" protocol="HTTP/1.1"
69 connectionTimeout="20000"
70 redirectPort="8443"
71 maxParameterCount="1000"
72 />
73 <!-- A "Connector" using the shared thread pool-->
74 <!--
75 <Connector executor="tomcatThreadPool"
76 port="8880" protocol="HTTP/1.1"
77 connectionTimeout="20000"
78 />
```

Scroll down to the connector port section and let's edit the response to mention it is a web server.



```
41 <Resource name="UserDatabase" auth="Container"
42 type="org.apache.catalina.UserDatabase"
43 description="User database that can be updated and saved"
44 factory="org.apache.catalina.users.MemoryUserDatabaseFactory"
45 pathname="conf/tomcat-users.xml" />
46 </GlobalNamingResources>
47
48 <!-- A "Service" is a collection of one or more "Connectors" that share
49 a single "Container" Note: A "Service" is not itself a "Container",
50 so you may not define subcomponents such as "Valves" at this level.
51 Documentation at /docs/config/service.html
52 →
53 <Service name="Catalina">
54
55 <!-- The connectors can use a shared executor, you can define one or more named thread pools-->
56 <!--
57 <Executor name="tomcatThreadPool" namePrefix="catalina-exec-"
58 maxThreads="150" minSpareThreads="4"/>
59 →
60
61 <!-- A "Connector" represents an endpoint by which requests are received
62 and responses are returned. Documentation at :
63 Java HTTP Connector: /docs/config/http.html
64 Java AJP Connector: /docs/config/ajp.html
65 APR (HTTP/AJP) Connector: /docs/apr.html
66 Define a non-SSL/TLS HTTP/1.1 Connector on port 8880
67 →
68 <Connector port="8880" protocol="HTTP/1.1"
69 connectionTimeout="20000"
70 redirectPort="8443"
71 maxParameterCount="1000"
72 Server="web server"
73 />
74 <!-- A "Connector" using the shared thread pool-->
75 <!--
76 <Connector executor="tomcatThreadPool"
77 port="8880" protocol="HTTP/1.1"
78 connectionTimeout="20000"
79 redirectPort="8443"
80 maxParameterCount="1000"
81 />
82 →
83
```

Add the line **Server = " web server"** under the connector port as shown above

save the information and quit.

Now let's restart the tomcat server. Using the command:

sudo systemctl start tomcat

Go back to Eclipse and execute the SimpleSocketClient program and record the response from the server.

Problems Javadoc Declaration Console X

SimpleSocketClient [Java Application] /opt/eclipse/eclipse/plugins/or
Loading contents of the URL: localhost
HTTP/1.1 200
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked
Date: Wed, 14 Jun 2023 01:23:34 GMT
Server: web server
2000